

Experimental Evaluation and Optimization of Real-Time Scheduling on STM32

Vladimir Toporkov
Computer Science and Engineering
Innopolis University
Innopolis, Russia
v.toporkov@innopolis.university

Artem Burmyakov
Institute of Information Security
Innopolis University
Innopolis, Russia
a.burmyakov@innopolis.ru

I. INTRODUCTION

A. Contribution

Our contributions are about evaluation of schedulers for STM32 microcontroller::

- we first implement Super Loop scheduler,
- then we implement the Earliest-Deadline First (EDF) scheduler, and finally
- we evaluate their performance upon a real STM32 hardware.

II. BACKGROUND ON STM32

The experimental platform consists of a uniprocessor STM32F767 microcontroller and external sensors connected through its hardware peripherals, as shown in Fig. 1.

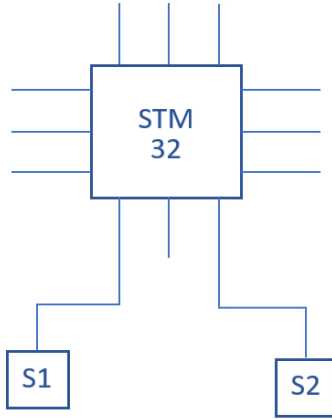


Fig. 1. STM32 hardware platform and connected sensors.

The hardware components are presented in Table I.

TABLE I
EXPERIMENTAL HARDWARE PLATFORM.

Component	Purpose
STM32F767	Scheduling and task execution
DHT11	Temperature and humidity measurement
HC-SR04	Distance measurement
OV2640	Image capturing

III. SYSTEM MODEL

1.AB@VT: To introduce necessary definitions and math notation. Use this notation later in Section IV.

- task;
- task set;
- mode 1 of a schedule, including absolute release time, completion time, etc.
- preemption;

Here, C_i , T_i , and D_i denote execution time, period and relative deadline respectively.

IV. SCHEDULING PRINCIPLES

A. Super Loop Scheduler

2.AB2VT: Here describe the pseudo-code of a Super Loop scheduler.

Let us first consider an example of a Super Loop schedule for two periodic tasks with implicit deadlines ($D_i = T_i$).

TABLE II
TASK SET USED IN THE SUPER LOOP EXAMPLE.

Task	C_i	T_i	D_i
τ_1	20	50	50
τ_2	10	20	20

In the considered Super Loop implementation, ready tasks execute in a fixed order and each selected job runs to completion.

The corresponding scheduling state is shown in Table III.

Let R_i^k denote the scheduled release time of job J_i^k and t_i^{next} the next release time tracked for task τ_i . At job completion, t^{cur} denotes current system time, a deadline miss is recorded if $t^{cur} > R_i^k + D_i$. If $R_i^j < t^{cur}$, the given release time is skipped and t_i^{next} is incremented again. A release scheduled for $t_i^{next} = t^{cur}$ is retained.

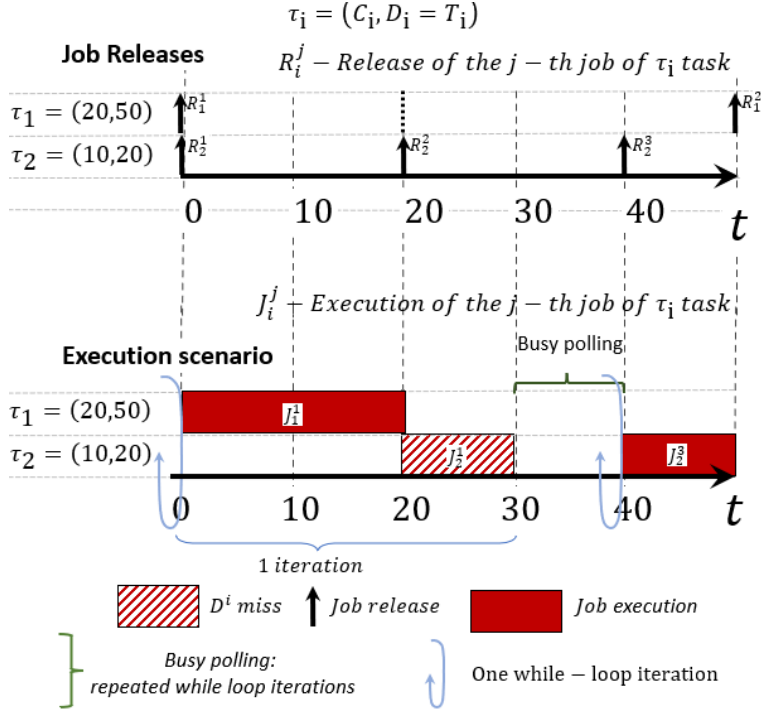


Fig. 2. Super Loop execution scenario for the task set in Table II.

TABLE III
SUPER LOOP SCHEDULING STATE FOR FIG. 2. RELEASE TIMES ARE SHOWN AFTER THE CHECKS AT EACH INSTANT.

t^{cur}	t_1^{next}	t_2^{next}	Deadline miss check	Release skip check
0	0	0	–	–
20	50	0	$20 < R_1^1 + 50$	$20 < R_1^2$
30	50	40	$30 > R_2^1 + 20$	$30 > R_2^2$ $30 < R_2^3$
40	50	40	–	–
50	50	60	$50 < R_2^3 + 20$	$50 < R_2^4$

In Fig. 2, J_1^1 executes from $t = 0$ to $t = 20$. Then, J_1^2 executes until $t = 30$, missing its deadline at $t = 20$. At completion, the release scheduled for $R_2^2 = 20$ is counted as skipped and t_2^{next} advances to $R_2^3 = 40$. Thus, no J_2^2 is executed.

B. Earliest Deadline First

3.AB2VT: To describe pseudo-code of EDF

The Super Loop mechanism does not take deadlines into account, jobs are selected according to a fixed loop order rather than their deadlines. As shown in Fig. 2, this may delay jobs beyond their deadlines. The considered implementation skips releases according to the rule above.

We consider preemptive EDF, which dynamically selects the ready job with the earliest absolute deadline. For a job J_i^k released at R_i^k , its absolute deadline is

$$d_i^k = R_i^k + D_i$$

If a newly released job has an earlier deadline than the running job, the running job is preempted.

V. SCHEDULERS IMPLEMENTATION FOR STM32

ToDo (code?)

VI. PERFORMANCE EVALUATION

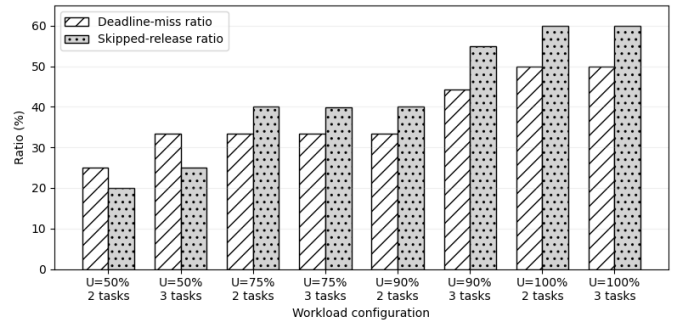


Fig. 3. Per-task deadline-miss and skipped-release ratios under Super Loop. Each configuration specifies the task-set utilization U and the number of tasks.